

Emily's Guide to Lightsheet Analysis
Last updated: Aug 19th 2026

Table of Contents

– Pipeline Basics –

Section 1: Pipeline Prep

Part 1: Downloading from the lightsheet

Part 2: Preparing your environment

Part 3a: Setting up your configuration file (config.yml) - adding a new cell marker

Part 3b: Setting up your configuration file (config.yml)

Section 2: Image preprocessing

Part 1: Image sorting & compression

Section 3: Segmentations

Part 1: Tissue Segmentation

Part 2: Cell Segmentation

Section 4: Post-segmentation processing

Part 1: Removing Sedimentation

Part 2: Move NE to Mesenchyme

Part 3: Landmarking

Part 4: Cell Density Maps

Part 5: Registration

Section 5: Analyses

Part 1: PCA

Part 2: Max/min Intensity Images

– Misc. –

Section 6: Using the pipeline on ARC

Section 7: Training your Cellpose model

Section 8: Correcting for antibody bleed down

Part 1: Correcting shift

Part 2: Antibody correction

Section 1: Pipeline Prep

All associated code can be found at: <https://github.com/emily-gv/LSFM-Pipeline-Mar2026.git>

Code can be run locally to test that everything is working, but batch analysis should be done using a compute cluster. Some steps can take 30hrs or upwards of 750GB RAM to run and your computer will be incapacitated.

You can run the pipeline in phases by commenting/uncommenting samples to be worked with & flags to use in the configuration file (further explained below).

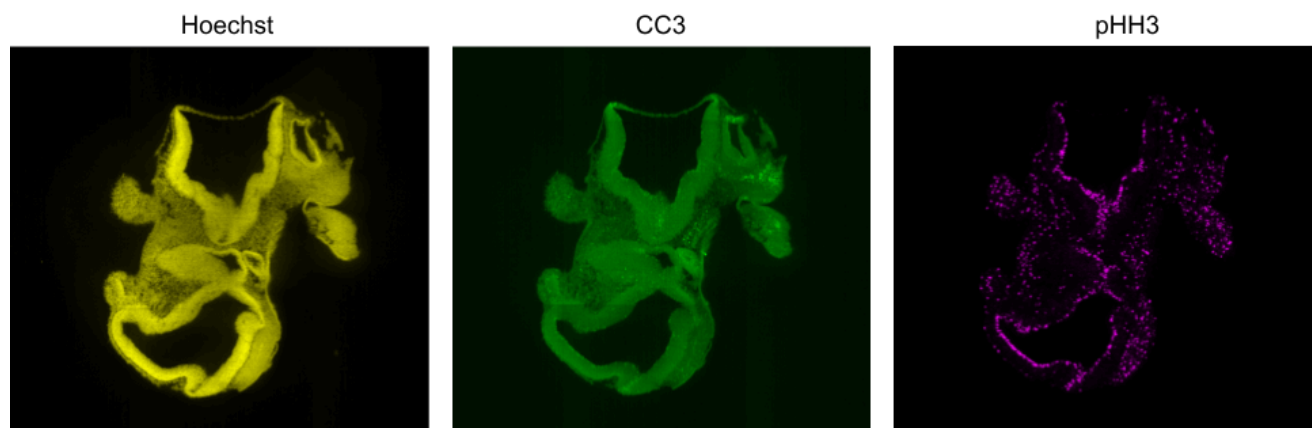
Prerequisites:

- A computer running **Linux**
- Git
 - If you don't have already it, run in terminal: `sudo apt install git`
- Anaconda/miniconda: <https://www.anaconda.com/download>
- Models to segment your cell markers of choice
 - Currently available models: tissue segmentation (mesenchyme + NE), nuclear (Hoechst & CSGreen validated - high intensity LSFM images), pHH3, CC3
 - To train your own model, see **Section 6**

Prep: Clear and stain samples following the lab's modified CUBIC protocol

Part 1: Downloading from the lightsheet

1. Manually adjust the channel intensities to minimize colour in the background while retaining the whole sample
 - a. Note: the currently trained CC3 model tolerates background within and behind the sample (see examples below)



2. Download your files as a z-stack
 - I kept them on the NAS, and then uploaded them to ARC (or downloaded locally) when ready to work with the compressed versions

On Zen Blue, select 'Method: Image Export' with the following parameters

- File Type: 'Tagged Image File Format (TIFF)'
- NO CLICK: 'Burn-in Annotations'
- CLICK: 'Short Format'
- NO CLICK: 'Use Channel Names'

Part 2: Preparing your environment

1. Clone the github repo
 - <https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository>
2. In terminal, navigate to the folder 'LSFM-Pipeline-Mar2026'

3. Run:


```
conda env create -f lsfm_pipeline.yml
```

 - Activate the environment using: `conda activate lsfm-pipeline`
4. With the environment activated, run:


```
conda install -c nvidia cuda-toolkit=12.2
```

```
conda install -c conda-forge libxcb
```

You are now ready to begin processing the tiff stacks.

Part 3a: Setting up your configuration file (config.yml) - adding a new cell marker (config.yml)

1. Under the 'markers' section (line 42), add a new entry corresponding to your marker. It should look like:


```
- name: 'marker-name'
  window_cellpose: 1024 (model-specific, see Section 6)
  flag_compress: False
  flag_segment: False
  flag_mesenchyme: False
  flag_remove_sedimentation: False
```
2. Uncomment and copy everything in the section at the top and paste it at the bottom of the file (or go line by line and paste your new versions under with their cc3/phh3 counterpart lines if you want to keep it clean and nice)
 - **IMPORTANT**: when replacing the marker name ensure capitalization is consistent with how you wrote it in Step 1

```
- folder_marker-name_tiles: "Step11_marker-name_tiles/Step12_marker-name_tiles"
- folder_marker-name_segmented_tiles_label: "Step22_marker-name_tiles_label"
- folder_marker-name_segmented_tiles_binary: "Step22_marker-name_tiles_binary"
- folder_marker-name_slices_label: "Step25b_marker-name_slices_label"
- folder_marker-name_slices_binary: "Step25b_marker-name_slices_binary"
- folder_marker-name_masked_label: "Step25b_marker-name_masked_label"
- folder_marker-name_masked_binary: "Step25b_marker-name_masked_binary"
- dest_file_marker-name_label_tiff: "_Step26_marker-name_label_raw.tiff"
- dest_file_marker-name_binary_tiff: "_Step26_marker-name_binary_raw.tiff"
- dest_file_marker-name_label_tiff_maskout_sedimentation: "_Step33_marker-name_label.tiff"
- dest_file_marker-name_binary_tiff_maskout_sedimentation: "_Step33_marker-name_binary.tiff"
```
3. Go to file AuxFunctions/sort_files.py. Add a **comma** on line 28 and uncomment line 29, adding your marker name (ensure capitalization is consistent with before)

Part 3b: Setting up your configuration file (config.yml)

1. Fill in 'samples' with your sample details

- folder_slices: full path to the folder of slices (can be NAS)
 - resX/Y/Z should be taken from Zen Black (Zen Blue doesn't give the precise resolution). Open your file and go to "Info"
 - Add a new line if necessary: subfolder_new-marker
2. Fill in the following with the full filepaths (stored locally):
 - folder_output
 - folder_CNN_architectures
 3. Fill in the following with the file names: (add your new cell marker if necessary)
 - filename_CNN_tissues
 - filename_CNN_nuclei
 - filename_CNN_cc3

Section 2: File Preprocessing

**** Make sure you have the conda environment activated****

Part 1: Image sorting & compression

1. Go to AuxFunctions/sort_files.py and check that the c1/c2/c3.tif (under output_folders, line 25) correspond to the correct channels/markers from the lightsheet
2. In config.yml, for all the marker channels you want to compress under 'markers' set 'flag_compress: True'
 - If you download a new sample everything might be set to True, but this way you can save time if you go back and adjust/re-download a single channel
3. In terminal, run: `python sort_and_compress.py`

After this step you are ready to begin segmenting the images.

Section 3: Segmentations

**** Make sure you have the conda environment activated****

If running this on ARC, for each sample you run, you'll need a copy of the 'config.yml', 'main_01/02.py', and the slurm file. Update the filenames accordingly within each: sample details in the config, 'config_filename' in main_01/02, and the python script in the slurm file.

Part 1: Tissue Segmentation

Try following the steps to run as-is. If it crashes severely, try uncommenting and editing the following lines in main_01_tissue_segmentation.py:

- `os.environ['XLA_FLAGS']`
- `os.environ['PATH']`
- `os.environ['LD_LIBRARY_PATH']`

1. Ensure you have samples commented/uncommented in config.yml as desired
2. In terminal, run: `python main_01_tissue_segmentation.py`
 - a. From the LFSM-Pipeline-Mar2026 folder, `sbatch ARC/main_01_arc.slurm` on ARC

Part 2: Cell Segmentation

1. For each marker you want to segment, toggle 'flag-segment: True' under 'markers' in config.yml
2. If you are interested in tracking cells in the edge of the mesenchyme, toggle 'flag_mesenchyme: True'
3. In terminal, run: `python main_02_tissue_segmentation.py`
 - a. `sbatch ARC/main_02_arc.slurm` on ARC

**** Nuclear cell segmentations may take upwards of 40hrs to run ****

- Also note you'll need to have the folder of tiled Hoechst images before running main_02 for nuclei



As-is main_02 requires the folder of tissue slices for the same sample. In the case you want to run main_02, but have deleted this folder (it takes up a lot of storage), comment out the line under "GENERATING TISSUE STATS".

By the end of this section you should have:

- a. TIFF volumes of the mesenchyme, NE, and mixed tissue
- b. GAN-related TIFF volumes
- c. TIFF volumes of your markers of choice (binary and label versions)
 - Binary is just your black and white segmentations
 - Label is Cellpose's attempt at also categorizing # of cells and other things
 - It's not too accurate in my experience. If you want to identify individual cells in the image I'd recommend using SciPy's label function (`from scipy.ndimage import label`)

Section 4: Post-segmentation processing

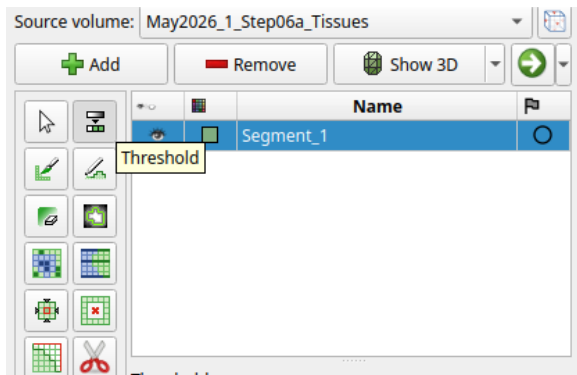
Prerequisites:

- 3DSlicer
- Paraview (recommended for visualization)
- XNView Multi Platform (recommended for visualization)

Part 1: Removing sedimentation

1. Begin by opening the file '_Step6a_Tissues.tiff' for one of your segmented samples
2. Navigate to the Segment Editor module, then click 'Add' to add a new segmentation

- Click the Threshold widget, then adjust the Threshold Range so that all tissues flash green (min 40-100 max works)



- Click 'Show3D' and center the image using the blue box buttons. Enlarge the blue box and you can use the scissor tool to cut around and remove any sedimentation.
- For more precise correction, use the eraser tool on one of the red/green/yellow box images (be sure to turn off Show3D before you do this for it not to lag)

Help, my computer keeps crashing!

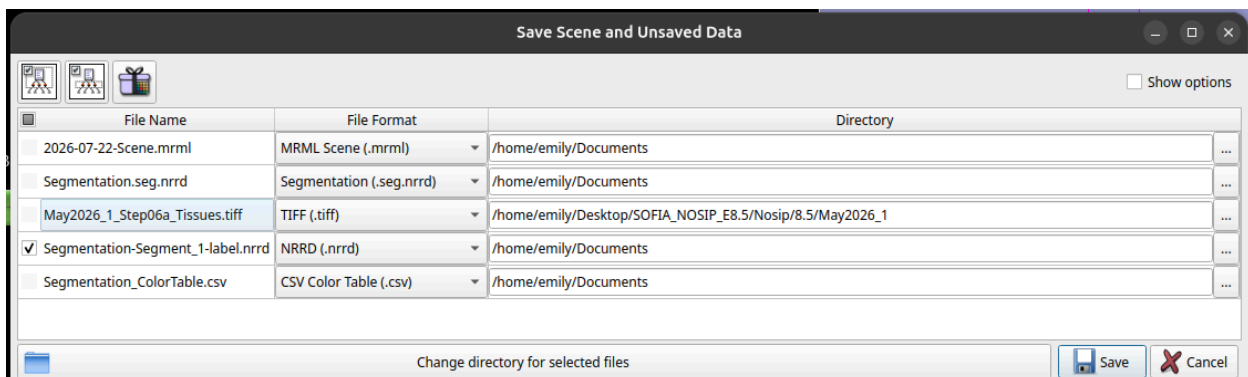


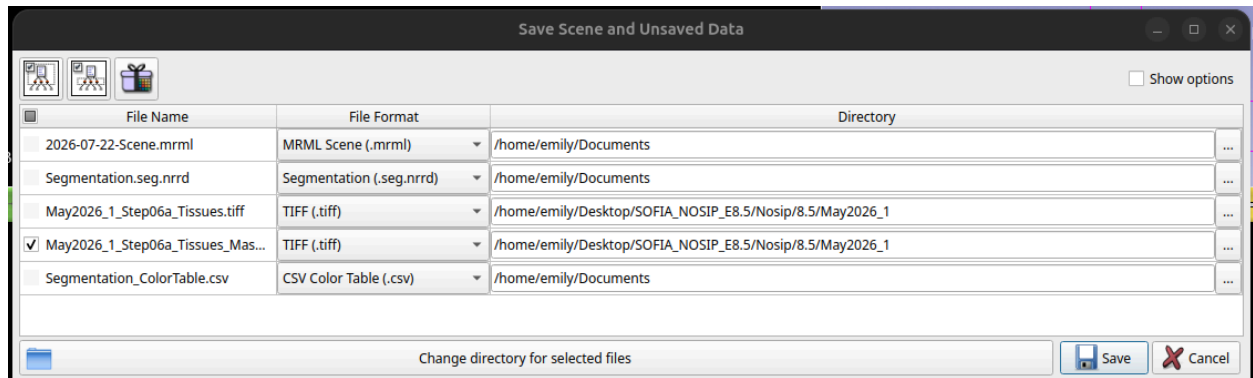
The TIFF likely has a smaller z step size, so it's a significantly larger file and uses up all your RAM. In addition to adding extra swap space, you can:

Open 3DSlicer from scratch and navigate to Segment Editor. Open python console with CTRL + 3, then type the following command:

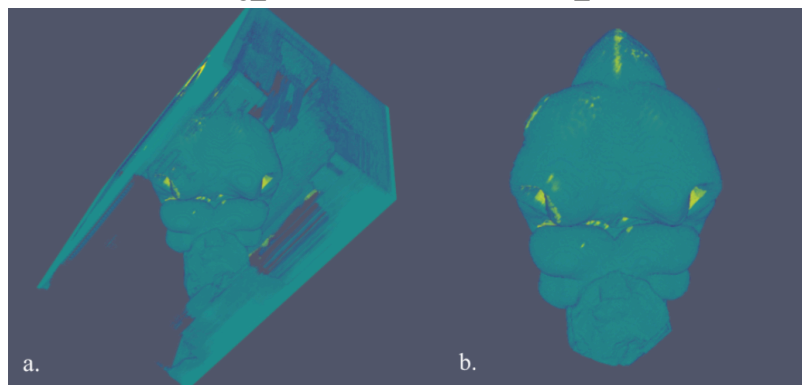
```
slicer.modules.SegmentEditorWidget.editor.setMaximumNumberOfUndoStates(2)
```

- Once finished with corrections, navigate to the module 'Data' and right click on the line 'Segment_1'. Select 'export visible segments to binary labelmap'
- Click the 'Save' icon at the top of the screen. Update the name/file extension & path for the .nrrd file. Rename it to **<sample-name>_Step06a_Tissues_Mask.tiff**. The file format drop down should adjust automatically. Save only that file.



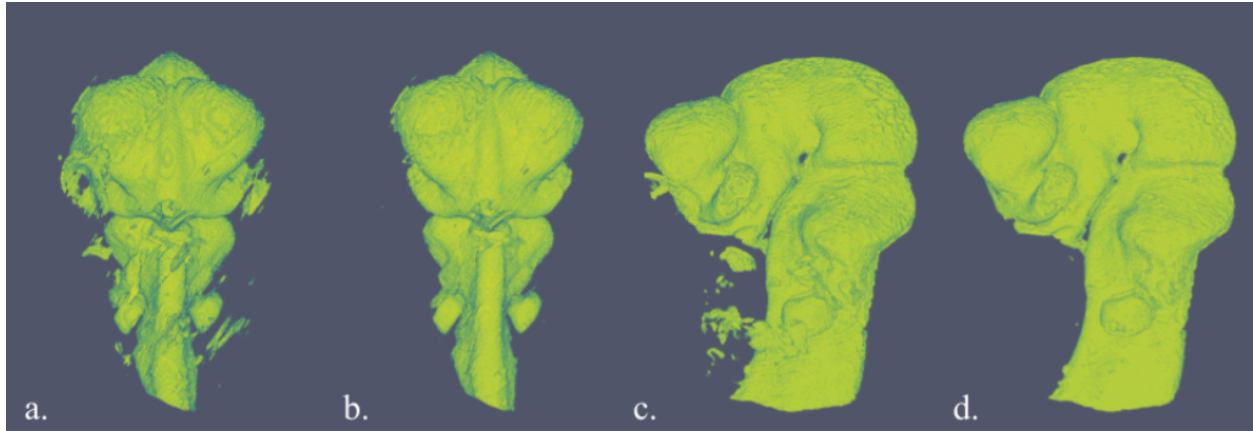


8. For all markers you're working with and have TIFF files for, toggle 'flag_remove_sedimentation: True' in the config.yml file
9. In terminal, run `python main_03_remove_sedimentation.py`
 - If you want to re-run this for just a cell marker and NOT the tissue files, you can toggle 'flag_tissue = False' in the main_03 file



Part 2: Move NE to Mesenchyme

1. Begin by opening the file 'xx_Step36a_NE.tiff' in 3DSlicer
2. Repeat steps 2-6, removing miscellaneous floating tissue at your own discretion (see example below)
3. Save the sample as '<sample>_Step36a_NE_corrected.tiff' (note the lowercase)
4. In terminal, run `python main_04_move_ne_to_mesenchyme.py`



Part 3: Landmarking

All my analyses were done using Chris Percival's 38 landmark set.

Paper: <https://pubmed.ncbi.nlm.nih.gov/25059626/>

Training video:

https://ucalgary.zoom.us/rec/share/AAyg9ez6HfK8acvr7GhmM0KRzPDwHyIqtBac2KweSGTY7qg6OgK3TujCaY04nArx.48UvKXBQkOaAem_Q

Passcode: X9LMQ=bR

There's also an excel of practical definitions for landmarking per embryonic stage.

1. Begin by opening the file 'xx_Step4a_Tissues.tiff' in 3DSlicer. Create a volume following the regular procedure in the Segment Editor module. Follow the training video and landmark your sample (create volume, create points list, landmark)
2. Export your landmarks: under 'Export/import Table' in the Markups module, select Output table: 'Create new table'. Select 'LPS' under Advanced. Click 'Export'.
3. Rename the .tsv to 'xx_38lms.csv' and save it to your sample's directory. Save only this file.
4. Open your csv file in a text editor, and rename your header ID, X, Y, Z, and get rid of everything after your columns of coordinates by using CTRL+F find and replace


```

ID,X,Y,Z
"F-1",377.336,820.213,886.644
"F-2",405.535,941.985,834.582
"F-3",473.14,1015.94,760.099
"F-4",533.983,1039.23,708.219
"F-5",643.91,1008.2,635.624
"F-6",430.464,1042.08,601.787
"F-7",330.263,886.342,503.508
"F-8",430.155,700.591,516.657
"F-9",490.868,718.027,438.57
"F-10",501.841,747.625,403.111
"F-11",524.172,783.518,385.671

```

Part 4: Cell Density Maps

At its max, running this for a single sample required 1000GB of RAM and 24hrs on ARC to run. This depends, of course, on the size of your samples. Some other samples were ~100GB for only a few hours. General rule of thumb was that if it crashed 3DSlicer I'd need a very large amount of memory to run the sample. There's probably some sort of memory optimization edits you could try on the scripts? I tried edits to delete intermediate matrices as it went along and such but it didn't help much.

Files you'll need to upload:

- '_Step30a_Tissues.tiff'
- '_Step33_{marker}_binary.tiff'

Note: you'll have to upload them to the correct output path per sample (eg. OUTPUT/Nosip/10.5/Feb2026_5)

**** Again, on ARC you'll have to make copies of your scripts per marker per sample, or wait until one is done running and then edit your config to toggle the other marker ****

1. In your config file, toggle 'flag_cell_density: True' for your marker of choice
2. Run the .slurm file from LSFM-Pipeline-Mar2026 folder (ie sbatch ARC/main_05_arc.slurm)

Part 5: Registrations

The following describes registration to a reference sample. The code for creating and registering samples to atlas / performing groupwise registration will be in the repo but I have not used it.

1. Ensure **all** samples are listed and uncommented in your main config file
 - a. The flags on lines 176-183 will allow you to each registration step independently
2. Make sure the configuration file listed in script_register_to_reference.py is correct. In terminal, run `python script_register_to_reference.py` (re: conda env)
 - a. The output will be a series of nested folders Sim/Sim/Affine/Sin. All your relevant outputs will be in the Syn folder.

Section 5: Analyses

Part 1: PCA

1. Create a mask of your region of interest in the tissue of interest. Open the file “`*Step53b_{Tissues,NE,Mes}`” in 3D Slicer and follow the instructions from Section 4.1, cropping the mask to your ROI. Save it as something adequately descriptive (eg. `Nosip_10.5_Tissues_Mask_WholeHead.tiff`)
3. Run `python script_register_to_reference.py`. Update the following lines:
 - ‘folder’ on line 19 with the path to your Syn folder
 - ‘map_name’ on line 20, changing ‘cc3’ to whatever marker you’re using
 - ‘downscale_masked_values’ on line 28. pHH3 analysis used 10, my cc3 analysis used 3.
 - ‘subfolder_name’ on line 29. I try to keep it consistent with whatever I named my ROI mask
4. Run `python script_pca_on_proliferation.py`. Update the following lines:
 - ‘folder’, ‘map_name’, ‘downscale_masked_values’, ‘subfolder_name’, consistent with whatever you used in Step 4
 - ‘groups_dict’ with all your samples, including mirrored samples (“{name}_mirrored”)



Either `script_correlation_in_mask` or `script_pca_on_proliferation` will give you an error about a sample not being found. At this point, you’ll need to delete the file ‘{sample_info}_{marker}_density_histnorm.tiff’ from the Syn folder. I just make a folder on Desktop to transfer problem files into so that I can move them back and forth.

Part 2: Max/Min Intensity Projections

Section 6: Using the pipeline on ARC

1. Connect to ARC via SSH. In terminal, type: `ssh <login.lastname>@arc.ualgary.ca`
 - If you aren’t on campus wifi you will need a VPN:
https://rcs.ualgary.ca/Connecting_to_RCS_HPC_Systems
2. In the terminal (not on ARC), upload the code folder using the `scp` command. For example:
`scp -r /home/emily/Documents/LSFM-Pipeline-Mar2026/script_register_to_reference.py emily.garciavolk@arc.ualgary.ca:/home/emily.garciavolk/`
 - Replace the first part with your local filepath to the folder, and the second replace the name(s) with your login credentials

3. Follow this guide to set-up conda on ARC:
https://rcs.ucalgary.ca/Conda_on_ARC#Creating_Conda_environments. Use the file tensorflow-20250512.yaml. If that doesn't work, try environment-tensorflow-cellpose-arc.yaml.
4. Within the LFSM-Pipeline-Mar2026 folder, create a subfolder called 'ARC'. This will house your SLURM scripts. Anytime you want to run a script, run ARC/xyz.slurm (the one in the repo comes set up with the ARC format for including conda in a job) from the LFSM-Pipeline-Mar2026 folder.
 - I recommend running one sample at a time to test out how time/storage requirements compare to the limits on your server. For each job you submit, you will need to copy the files you use and adjust the files accordingly (depends, but I believe if you run script1.py using config1.yml and then adjust config1, if it's in queue script1 will end up running using the edited file. Save yourself the hassle of finding out by using separate files per sample).
 - For example:
 - main_01_tissue_segmentation_May2026_105_1.py
 - main_01_tissue_segmentation_May2026_105_2.py
 - config_May2026_105_1.yml
 - config_May2026_105_2.yml
 - main_02_cell_segmentation_May2026_1.py
 - main_02_cell_segmentation_May2026_2.py

If you're not at UCalgary, find your respective university's server guide and set up the conda environment following those instructions.

Section 7: Training Your Cellpose Model

1. Download lightsheet image slices of your marker of choice (normal colour is fine)
 - a. For reference, I used 22 slices across 3 samples, with a 50/50 training/validation split
2. Run AuxFunctions/grayscale.py
3. Crop your grayscale images to a subsection of 1024px by 1024px (I used GIMP)
4. Annotate image tiles
 - a. Training: annotate 11 image TILES from 2 samples
 - b. Validation: annotate 11 image TILES from 1 sample
5. To train the model, use the code found at
https://github.com/lucaslovecio/ACHRI_Workshop_Cellpose

Part 2: Metrics

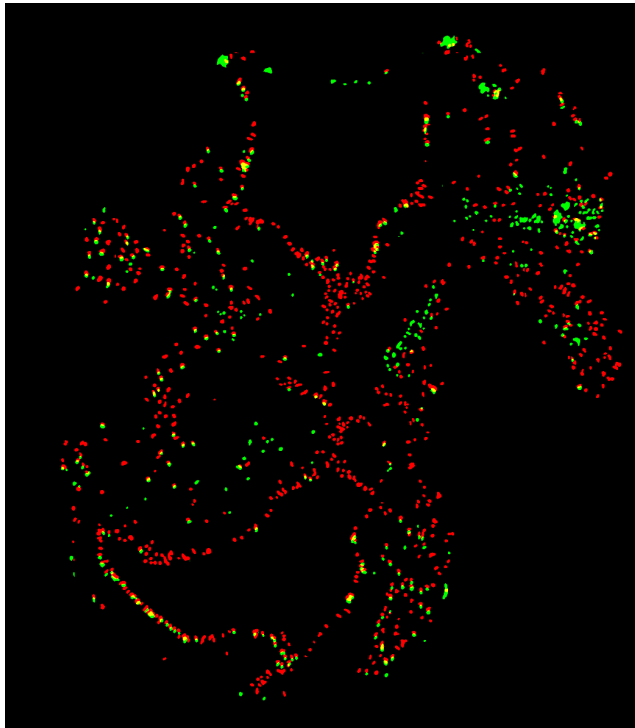
1. Prepare 4 folders: x_annotation, x_grayscale, x_masked, x_segmented
2. Tile grayscale slices of select marker channel and annotate, leave in respective folders (AuxFunctions/grayscale.py)
3. Run Cellpose/segment_folder_cellpose.py
4. Run Cellpose/createMask.py
5. Run Cellpose/computeMetrics_cellpose.py

Section 8: Correcting for Antibody Bleed Down

You can probably modify this code to just use the 3D tiff images, but for my ease of use I just used the folders of segmentations and then made a 3D tiff volume after correction.

Part 1: Correcting shift

1. Download your folders “Step25b_cc3_slices_binary” and “Step25b_phh3_slices_binary”
2. In file CorrectAntibody/compareOverlap.py, run either `compute_image_overlay` (single image) or `compute_image_overlay_folder` (whole folder). This will give you a merged image where one marker is in **red**, the other in **green**, and the overlap in **yellow**.



3. Open one of these merged images and manually count how many pixels you'd have to shift a cell to have complete overlap (as best as possible).
 - a. Kinda just use your best judgement as to which are actually the same cells
 - b. I used GIMP as it gives you pixel position for where you have your cursor placed
4. In `compareOverlap.py`, run `correct_shift`, using the pixel value you determined
 - You can then rerun `compute_image_overlay_folder` using the output folder to sanity check that you shifted the correct channel

Part 2: Antibody correction

1. In `CorrectAntibody/correct_cells.py`, run `remove_cells_folder_vs_folder` following the specifications outlined in the function comments
2. In `AuxFunctions/script_create_volume_from_tiffs.py`, set up `folder_input` as your folder of corrected cc3 binary images. If you've lost track of the phh3 tiff you can re-run it on your phh3 binary folder but you should already have the phh3 volume from `main_02`.
 - a. MAKE SURE to name your resulting volumes “_Step26_{cc3/phh3}_binary_raw.tiff” (and delete the original ones!), then move them into your pipeline output directory (eg. `Nosip/10.5/Aug27_1`) and then you can go straight into running `main_03_remove_sedimentation.py` and so forth